

After adding the test project to solution:

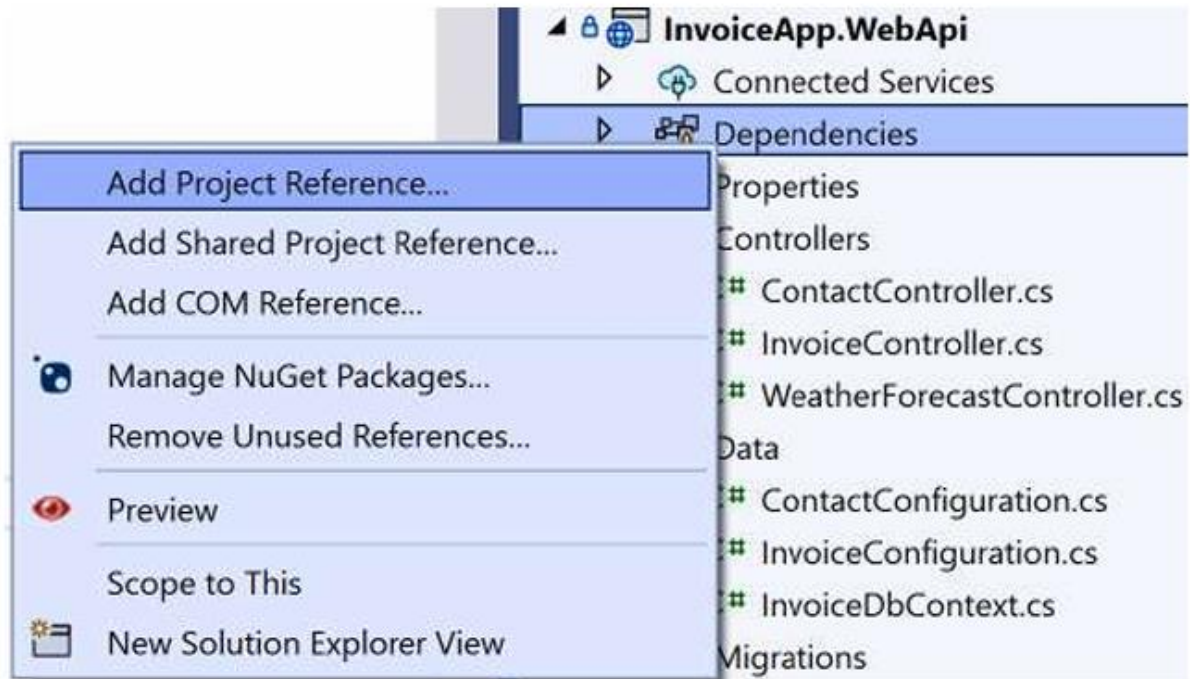


Figure 9.3 – Adding a project reference to the test project in VS 2022

If you use the .NET CLI, you can create a new xUnit test project by running the following commands in the terminal:

```
dotnet new xunit -n InvoiceApp.UnitTests
```

Then, you can add the test project to the solution by running the following commands:

```
dotnet sln InvoiceApp.sln add InvoiceApp.UnitTests/InvoiceApp.  
UnitTests.csproj
```

You also need to add the reference to the main project by running the following command:

```
dotnet add InvoiceApp.UnitTests/InvoiceApp.UnitTests.csproj reference  
InvoiceApp.WebApi/InvoiceApp.WebApi.csproj
```

The default xUnit test project template contains a sample unit test. You can delete the sample unit test named `UnitTest1.cs`; we will write our own unit tests in the next section.

If you use the .NET CLI, you can run the unit tests by running the following command in the terminal:

```
dotnet test
```

```
*****
```

The test class is any class that has [Fact]-Annotation:

```
using InvoiceApp.WebApi;
using InvoiceApp.WebApi.Models;
using InvoiceApp.WebApi.Services;

namespace JLoka.InvoiceApp.UnitTests
{
    public class EmailServiceTests
    {
        [Fact]
        public void TestGenerateEmail_Should_Return()
        {
            var invoiceDate = DateTimeOffset.Now;
            var dueDate = invoiceDate.AddDays(30);
            var invoice = new Invoice
            {
                Id = Guid.NewGuid(),
                InvoiceNumber = "INV-001",
                Amount = 500,
                DueDate = dueDate,
                Contact = new Contact
                {
                    Id = Guid.NewGuid(),
                    FirstName = "John",
                    LastName = "Doe",
                    Email = "john.doe@example.com"
                },
                Status = InvoiceStatus.Draft,
                InvoiceDate = invoiceDate,
                InvoiceItems = new List<InvoiceItem>
                {
                    new()
                    {
                        Id = Guid.NewGuid(),
                        Description = "Item 1",
                        Quantity = 1,
                        UnitPrice = 100,
                        Amount = 100
                    },
                    new()
                    {
                        Id = Guid.NewGuid(),
                        Description = "Item 2",
                        Quantity = 2,
                        UnitPrice = 200,
                        Amount = 400
                    }
                }
            };
        }
    }
}
```

```

        var (to, subject, body) = new
EmailService().GenerateInvoiceEmail(invoice);
        Assert.Equal(invoice.Contact.Email, to);
        Assert.Equal($"Invoice INV-001 for John Doe", subject);
        Assert.Equal($""
Dear John Doe,

Thank you for your business. Here are your invoice details:
Invoice Number: INV-001
Invoice Date: {invoiceDate.LocalDateTime.ToShortDateString()}
Invoice Amount: {invoice.Amount.ToString("C")}
Invoice Items:
Item 1 - 1 x $100.00
Item 2 - 2 x $200.00

Please pay by {invoice.DueDate.LocalDateTime.ToShortDateString()}. Thank
you!

Regards,
InvoiceApp
"", body);
    }
}
}

```

The output shows that the unit test passed. If you want to see the detailed test results, you can run the following command:

```
dotnet test --verbosity normal
```

You will see detailed test results that show the test name, the result, the duration, and the output.

To [Moq](#) the methods in [ASP.NET Core](#) we need module named: [Moq](#)

To Mock the functions of interface or virtual members:

```
[Fact]
public async Task TestSendEmail_SouldSend()
{
    // Arrange
    var to = "user@example.com";
    var subject = "Test Email";
    var body = "Hello, this is a test email";

    var emailSenderMock = new Mock<IEmailSender>();
    object t1 = emailSenderMock.Setup(m =>
m.SendEmailAsync(It.IsAny<string>(), It.IsAny<string>(),
It.IsAny<string>()))).Returns(Task.CompletedTask);
    //var loggerMock = new Mock<ILogger<IEmailService>>();
    //loggerMock.Setup(l => l.Log(It.IsAny<LogLevel>(), It.IsAny<EventId>(),
It.IsAny<It.IsAnyType>(),
//    It.IsAny<Exception>(), (Func<It.IsAnyType, Exception?,
string>)It.IsAny<object>()))).Verifiable();
    var emailService = new EmailService(emailSenderMock.Object);
    // Act
    await emailService.SendEmailAsync(to, subject, body);
    // Assert
    emailSenderMock.Verify(m => m.SendEmailAsync(It.IsAny<string>(),
It.IsAny<string>(), It.IsAny<string>()), Times.Once);
}
}
```

```
emailSenderMock.Setup(m => m.SendEmailAsync(It.IsAny<string>(),
It.IsAny<string>(), It.IsAny<string>()))
.ReturnsAsync(true);
```

Mocking the ILogger interface

The EmailService class uses the ILogger interface to log the information and errors. We use the LogInformation() method to log the information and LogError() method to log the errors. However, we cannot mock the LogInformation() or LogError() method directly because they are extension methods on top of the ILogger interface. These extension methods, such as LogInformation(), LogError(), LogDebug(), LogWarning(), LogCritical(), LogTrace(), and others, all call the Log() method of the ILogger interface. Therefore, to verify that a given log message is logged, it is necessary to mock only the Log() method of the ILogger interface.

If the `SendEmailAsync()` method throws an exception, we need to ensure the logger will log the exception when the exception occurs. To test the failure scenario, we need to mock the `SendEmailAsync()` method so that it throws an exception. We can use the `ThrowsAsync()` method to mock the `SendEmailAsync()` method to throw an exception explicitly. The following code shows how to mock the `SendEmailAsync()` method to throw an exception:

```
emailSenderMock.Setup(m => m.SendEmailAsync(It.IsAny<string>(),
    It.IsAny<string>(), It.IsAny<string>()))
    .ThrowsAsync(new SmtpException("Test SmtpException"));
```

Then, we can verify whether the `LogError()` method of the `ILogger` interface is called when the `SendEmailAsync()` method throws an exception, as shown here:

```
// Act + Assert
await Assert.ThrowsAsync<SmtpException>(() => emailService.
    SendEmailAsync(to, subject, body));
```

To install `FluentAssertions`, we can use the following .NET CLI command:

```
dotnet add package FluentAssertions
```

Let's see how to verify the exception using `FluentAssertions`. In the `EmailServiceTests` class, there is a `SendEmailAsync_Should_Log_SmtpException()` method. This method verifies whether the `SendEmailAsync()` method will log the exception when the `SendEmailAsync()` method throws an exception. The following code shows how to use `xUnit` to verify the exception:

```
await Assert.ThrowsAsync<SmtpException>(() => emailService.
    SendEmailAsync(to, subject, body));
```

We can use the `Should().ThrowAsync<>()` method of `FluentAssertions` to verify the exception, as shown in the following code:

```
var act = () => emailService.SendEmailAsync(to, subject, body);
await act.Should().ThrowAsync<SmtpException>().WithMessage("Test
    SmtpException");
```

You can even enrich the error message by adding a custom message to the assertion method. For example, we can add a custom message to the `Should().Be()` method, as follows:

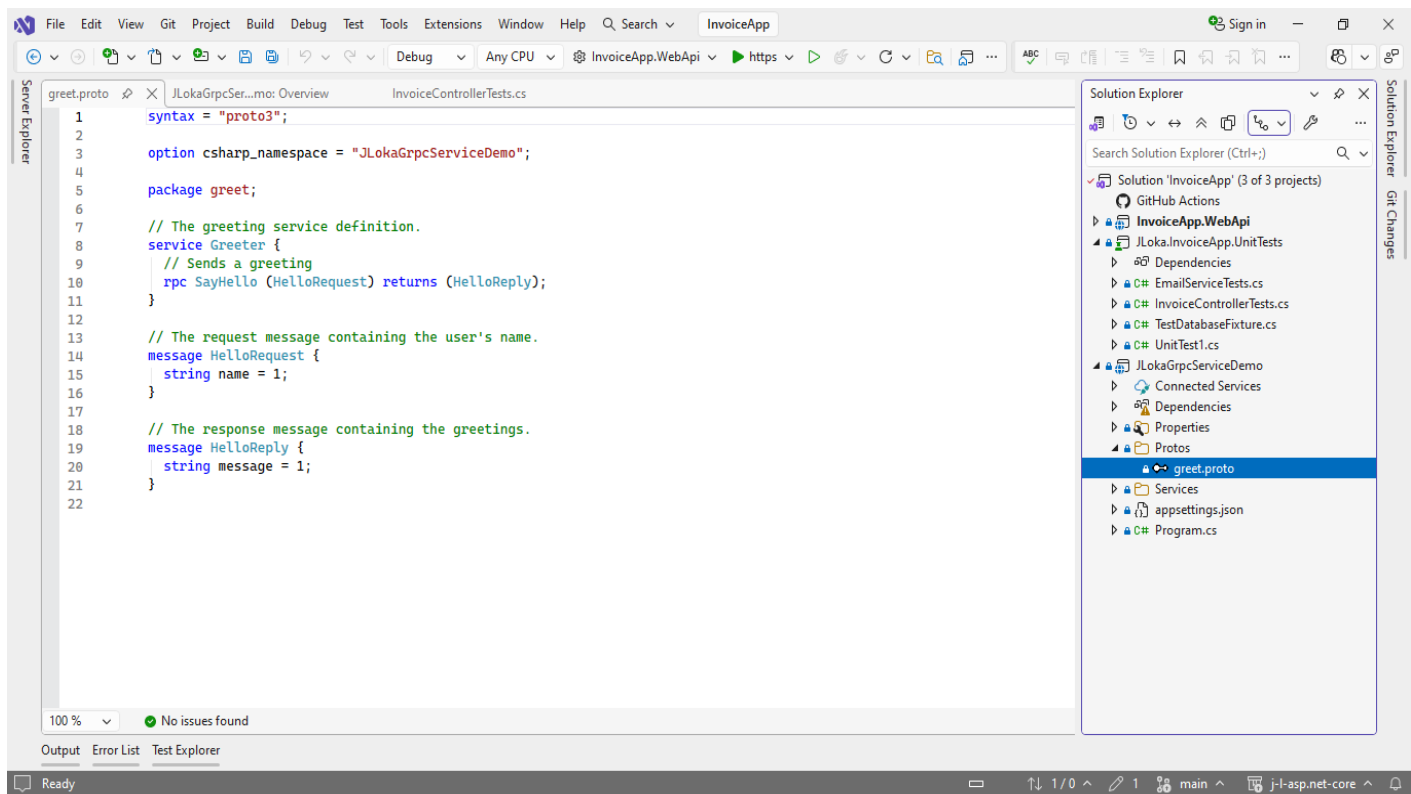
```
returnResult.Count.Should().Be(3, "The number of invoices should be 3");
```

Now, the error message will be as follows:

```
Expected returnResult.Count to be 3 because The number of invoices should be 3, but found 2.
```

Therefore, it is highly recommended to use `FluentAssertions` in your tests. It makes your tests more readable and maintainable.

To create new gRPC service from visual studio:



The `greet.proto` file is the `proto` file that defines the `gRPC` service and `messages`.

It is the contract of the `gRPC` service. In the preceding `proto` file, we define a service named `Greeter` with a method named `SayHello()`. The `SayHello()` method takes a `HelloRequest` message as `input` and returns a `HelloReply` message as `output`. Both `HelloRequest` and `HelloReply` messages have string properties named `name` and `message`, respectively.

We need to check if `Google.Protobuf` is installed or not, it provides runtime for `Protobuf`

The basic service to run with `protos` is:

```
using Grpc.Core;
using JLokaGrpcServiceDemo;

namespace JLokaGrpcServiceDemo.Services
{
    public class GreeterService : Greeter.GreeterBase
    {
        private readonly ILogger<GreeterService> _logger;
        public GreeterService(ILogger<GreeterService> logger)
        {
            _logger = logger;
        }

        public override Task<HelloReply> SayHello(HelloRequest request,
ServerCallContext context)
        {
            return Task.FromResult(new HelloReply
            {
                Message = "Hello " + request.Name
            });
        }
    }
}
```

The service binding for app:

```
using JLokaGrpcServiceDemo.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddGrpc();

var app = builder.Build();

// Configure the HTTP request pipeline.
app.MapGrpcService<GreeterService>();
app.MapGet("/", () => "Communication with gRPC endpoints must be made through a gRPC
client. To learn how to create a client, visit:
https://go.microsoft.com/fwlink/?linkid=2086909");

app.Run();
```

```
message HelloRequest {
    string name = 1;
    string address = 2;
}
```

Save the file and go back to the `GreeterService` class. In the `SayHello()` method, you can try to access the `Address` property of the `HelloRequest` object. You will find that the `Address` property is not available. This is because the generated code is not updated. We need to regenerate the code by using the `dotnet build` command. Alternatively, you can delete the `obj` folder and the code will be regenerated automatically.

gRPC is a **contract-first framework**, meaning that the **gRPC service** and **messages** must be defined in a **proto file**. When we talk about **messages**, we are talking about the data that is sent **between the client and the server**. While **gRPC messages** may be similar to the data model in RESTful APIs, they are not the same. RESTful APIs are centered around **resources**, and the **data model** is usually a **resource model** that can be mapped to one or **multiple database tables**. In contrast, **gRPC is action-based**, and the message can be any other type of **data model** or **other message** sent between the **client** and the **server**. Therefore, **gRPC messages** may not be exactly mapped to a resource model in RESTful APIs.

the `option csharp_namespace` attribute is only used by the C# code generator.

`proto3` does not support `GUID` and `decimal` types.

```
syntax = "proto3";

option csharp_namespace = "JLokaGrpcServiceDemo.Protos";

message CreateAddressRequest {
    string street = 1;
    string city = 2;
    string state = 3;
    string zip_code = 4;
    string country = 5;
}
```

The `numbers` following the field names are called `field numbers`. Field numbers play an `important` role in the `proto file`. These field numbers are `used to identify the fields in the message`.

By using field numbers instead of field names when encoding the message, `we can make the gRPC message more compact`. This is because numbers `are shorter than` field names. Additionally, `protobuf uses a binary format`, which is more compact than plain text formats such as `JSON` and `XML`. This further helps reduce the size of the message.

There are a few things to note about field numbers according to the `protobuf documentation`:

- The range of field numbers is from 1 to 536,870,911. So we cannot use 0 as a field number.
- The field numbers must be unique within a message.
- Field numbers 19000 to 19999 are reserved for `protobuf`, so you cannot use them.
- Technically, the order of the field numbers does not matter. It is recommended to use the ascending order of the field numbers. Smaller field numbers use fewer bytes to encode. For example, a field number between 1 and 15 uses only one byte to encode, but numbers from 16 to 2047 use two bytes.

- Once a field number is assigned to a field, it cannot be changed if the proto file is used in production. Changing a field number will break the backward compatibility of the proto file.
